

File Operations

CHAPTER 10 - FILE COPYING IN PYTHON

1. What is File Copying ?

File copying means creating a duplicate of a file from one location to another location. The content of the source file is read and written into the destination file.

2. Why do we need File Copying ?

- To create a **backup** of important files.
- To **transfer** files from one location to another.
- To **duplicate** files for **safe storage**.
- To work on a copy **without affecting** the original file.

3. Types of File Copying

1. **Copying Text Files** → Files that contain text (.txt, .py, .csv, etc.)
2. **Copying Binary Files** → Files that contain binary data (images, videos, pdf, audio, etc.)

4. Copying a Text File

Program : Copying a Text File

```
# source file (source.txt)
with open("source.txt", "r") as src:
    data = src.read() # read all data

# destination file (destination.txt)
with open("destination.txt", "w") as dest:
    dest.write(data) # write data

print("File copied successfully.")
```

Explanation :

1. Open the source file in read mode ("r").
→ read() reads the entire content and stores it in variable data.
2. Open the destination file in write mode ("w").
→ write() writes the data to the new file.
3. If destination file does not exist, it will be created automatically.
4. If it exists, its previous content will be overwritten.

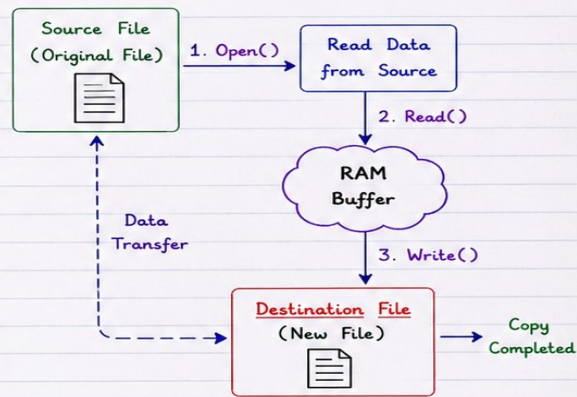
★ How it Works ?

1. Source file is opened in read mode. → Content is read and stored in memory (RAM).
2. Destination file is opened in write mode. → Content from memory is written into it.
3. After writing is complete, both files are closed. → Copy operation is finished.



Note : This method works well for small files. For very large files, reading whole content at once may use more memory.

How File Copying Works (Internal Working)



Only these create a new local scope:

- Functions (def)
- Methods
- Lambda expressions

```
with open("source.txt", "r") as src:
    data = src.read()
```

After this block finishes:

- src is closed automatically.
- data is not destroyed.

CHAPTER 10 - FILE COPYING IN PYTHON

5. Copying Line by Line (Text Files)

Why?

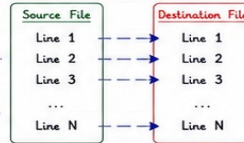
- For very large text files, reading entire content at once may use a lot of memory.
- Reading line by line is memory efficient.

Program : Copy Text File Line by Line

```
with open("source.txt", "r") as src:
    with open("destination.txt", "w") as dest:
        for line in src:
            dest.write(line) # write one line at a time
```

How it Works ?

- Open source file in read mode.
- Open destination file in write mode.
- Read one line from source file.
- Write that line into destination file.
- Repeat until end of file.



6. Copying Binary Files

Why Binary Mode ?

- Images, videos, pdf, audio, executables etc. contain binary data.
- Text mode can change data (e.g. newline\n conversion), so we must use binary mode.

Program : Copy Binary File

```
with open("image.png", "rb") as src:
    data = src.read() # read all bytes
with open("copy.png", "wb") as dest:
    dest.write(data) # write all bytes
```

Examples of Binary Files

- .png, .jpg, .jpeg (images)
- .mp4, .mkv (videos)
- .mp3, .wav (audio)
- .pdf, .docx, .exe, .rar, etc.

Text Mode vs Binary Mode (Important Difference)

Feature	Text Mode	Binary Mode
Opening Mode	"r", "w", "a", "r+", "w+", etc.	"rb", "wb", "ab", "rb+", "wb+", etc.
Data Type	str (string)	bytes
Newline Handling	Python converts newline (\n) automatically	No conversion, data is copied as it is
Best For	Text files (.txt, .py, .csv, .log)	Binary files (images, videos, pdf, audio, exe)
Size Change	Possible (due to newline conversion)	No change, exact copy
Example	with open("file.txt", "r")	with open("file.png", "rb")

7. Common Pattern for Binary Copy (Read All at Once)

```
with open("video.mp4", "rb") as src:
    data = src.read()
with open("copy_video.mp4", "wb") as dest:
    dest.write(data)
```

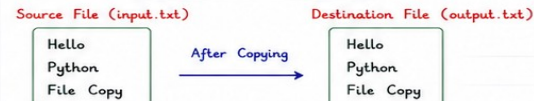
☆ **Note** : This method is simple but not suitable for very large files because it loads the entire file into memory.



Key Point :

Always use binary mode (rb, wb) for non-text files. Otherwise, the file may get corrupted.

Small Example (Text Copy)



CHAPTER 10 - FILE COPYING IN PYTHON

8. Copying Large Files Using Chunks

Why Chunks ?

- read() loads the entire file into memory.
- For very large files (GBs), this may cause high memory usage or even MemoryError.
- Reading and writing in small chunks is memory efficient.

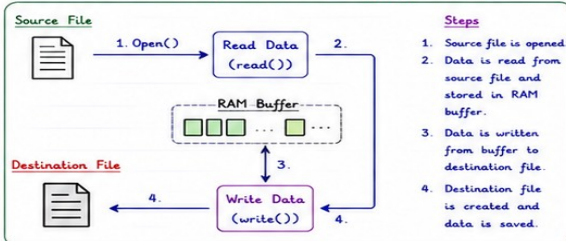
Program : Copy Large File Using Chunks

```
CHUNK_SIZE = 4096 # 4 KB
with open("large_file.zip", "rb") as src:
    with open("copy_large_file.zip", "wb") as dest:
        while True:
            chunk = src.read(CHUNK_SIZE) # read chunk
            if not chunk:                # no more data
                break
            dest.write(chunk)            # write chunk
```

How it Works ?

- ① Open source file in binary read mode.
- ② Open destination file in binary write mode.
- ③ Read a small chunk (e.g., 4 KB) from source.
- ④ Write the chunk to destination.
- ⑤ Repeat until no more data (read() returns empty bytes).
- ⑥ Copy operation completed.

9. Internal Working (How File Copying Works)



10. Common Mistakes

- ✗ Opening a binary file in text mode ("r" instead of "rb").
- ✗ Opening a text file in binary mode ("rb" instead of "r").
- ✗ Using read() for very large files.
- ✗ Forgetting to close files (use with statement).
- ✗ Overwriting the source file by giving same name for destination.

11. Best Practices

- ✓ Always use with statement for safety.
- ✓ Use binary mode (rb, wb) for non-text files.
- ✓ Use chunk method for large files.
- ✓ Handle exceptions while copying important files.
- ✓ Verify the destination file after copying.
- ✓ Keep backup of important files before copying.

Quick Syntax Reference

Text File Copy

```
with open(src, "r") as s:
    with open(dest, "w") as d:
        d.write(s.read())
```

Binary File Copy

```
with open(src, "rb") as s:
    with open(dest, "wb") as d:
        d.write(s.read())
```

12. Verify Copied File

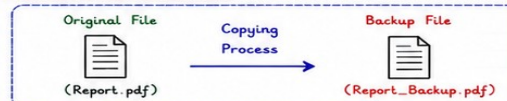
Program : Verify Using File Size

```
import os
src = "file.txt"
dest = "copy_file.txt"

if os.path.getsize(src) == os.path.getsize(dest):
    print("File copied successfully and sizes match.")
else:
    print("File copied but sizes do not match.")

# Works for both text and binary files
```

13. Real-Life Example



Key Takeaway



File copying is the process of reading data from a source file and writing it into another file. Use the right mode (text/binary) and the right method (normal/chunk) for safe and efficient copying.

Summary of Methods

Method	When to Use	Mode	Best For	Memory Usage
Simple Read & Write (read())	Small files	"r" / "w" or "rb" / "wb"	Small text/binary files	High (loads entire file)
Line by Line (for line in src)	Text files	"r" / "w"	Large text files	Low
Chunk Copying (read(size))	Very large files	"rb" / "wb"	Large binary files	Very Low

CHAPTER 11 - OS MODULE IN PYTHON

1. What is OS Module ?

The os module in Python provides a way to interact with the operating system. It allows us to perform various operations like managing files, directories, paths, environment variables, etc.

2. Why do we use OS Module ?

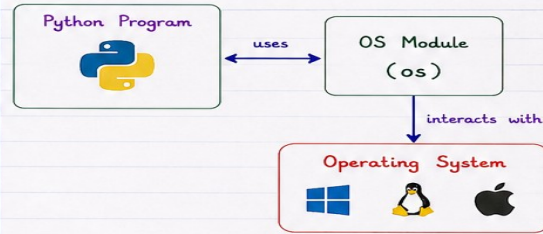
- To perform operations on files and directories.
- To get information about the system.
- To change the current working directory.
- To rename or delete files and directories.
- To check existence, size, path and other details.
- To create and remove directories.

3. Importing the OS Module

We can import the os module using the following statement:

```
import os # import entire os module
```

☆ **Note** : OS module comes built-in with Python. No need to install anything separately.



Common Uses of OS Module

- ✓ File and directory management
- ✓ Path related operations
- ✓ Environment variables
- ✓ Getting system information
- ✓ Process management

4. os.rename() - Rename a File or Directory

- The os.rename() function is used to rename a file or directory.
- If the destination already exists, it will be overwritten.
- It works for both files and directories.

Example 1 : Rename a File

```
import os
# before renaming
# file name : old.txt
os.rename("old.txt", "new.txt")
# after renaming
# file name : new.txt
```

Explanation :

The file 'old.txt' is renamed to 'new.txt' in the same directory.



Syntax

```
os.rename(src, dst)
src → current name (path)
dst → new name (path)
```

Example 2 : Rename a Directory

```
import os
# before renaming
# directory name : data
os.rename("data", "my_data")
# after renaming
# directory name : my_data
```

Explanation :

The directory 'data' is renamed to 'my_data'.



Key Point : Use os.rename() for renaming files or directories. It cannot be used to move a file/directory to another location.

5. os.remove() - Delete a File

- The os.remove() function is used to delete a file.
- If the file does not exist, it raises an error (FileNotFoundError).

Syntax

```
os.remove(path)
path → file path (string)
```

Example : Delete a File

```
import os
file_name = "old.txt"
os.remove(file_name)
# after deletion
# old.txt is removed from the directory
```

Explanation :

The file 'old.txt' will be deleted permanently.



Note : Delete operation cannot be undone. Use with care.

6. os.path.exists() - Check if Path Exists

- The os.path.exists() function checks whether a file or directory exists at the specified path.
- It returns True if the path exists, otherwise it returns False.

Syntax

```
os.path.exists(path)
path → file or directory path (string)
```

Example 1 : Check if File Exists

```
import os
path = "old.txt"
if os.path.exists(path):
    print("File exists.")
else:
    print("File does not exist.")
```

Output (Example)

If old.txt exists
↓
File exists.

Example 2 : Check if Directory Exists

```
import os
path = "data"
if os.path.exists(path):
    print("Directory exists.")
else:
    print("Directory does not exist.")
```

Output (Example)

If 'data' exists
↓
Directory exists.

Return Value

```
os.path.exists(path) → True → Path exists (file or directory)
                        → False → Path does not exist
```

Key Point :

This function works for both files and directories.

7. os.path.getsize() - Get Size of a File

- The os.path.getsize() function returns the size of the file in bytes.
- If the file does not exist, it raises an error.

Syntax

```
os.path.getsize(path)
path → file path (string)
```

Example : Get File Size

```
import os
path = "image.png"
size = os.path.getsize(path)
print(f"Size of the file: {size} bytes")
```

Output (Example)

Size of the file: 20480 bytes

Note :

1 KB = 1024 bytes
1 MB = 1024 KB

8. os.mkdir() - Create a Directory

- The os.mkdir() function is used to create a new directory.
- It creates only one level of directory.

Syntax

```
os.mkdir(path)
path → directory name (string)
```

Example : Create a Directory

```
import os
os.mkdir("my_folder")
print("Directory created successfully.")
```

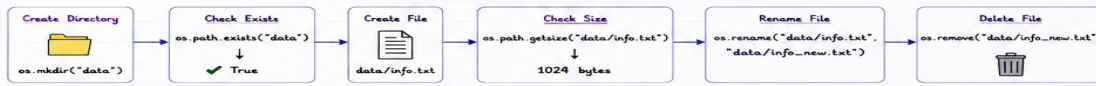
Output (Example)

Directory created successfully.

★ Important :

If the directory already exists, it raises FileExistsError.

Example : Complete Workflow



Key Takeaway :

The os module provides simple and powerful functions to interact with the operating system. Always check existence before performing operations to avoid errors.

CHAPTER 11 - OS MODULE IN PYTHON

9. os.rmdir() - Remove an Empty Directory

- The `os.rmdir()` function is used to remove an empty directory.
- It raises an error if the directory is not empty or does not exist.

Syntax

```
os.rmdir(path)
path → directory path (string)
```

Example : Remove an Empty Directory

```
import os
os.rmdir("empty_folder")
print("Directory removed successfully.")
```

Output (Example)
Directory removed successfully.



Important :

The directory must be empty. Use `os.remove()` to delete files, then use `os.rmdir()` to remove the directory.

10. os.listdir() - List Directory Contents

- The `os.listdir()` function returns a list of all files and directories in the given path.
- It does not include the path itself.

Syntax

```
os.listdir(path)
path → directory path (string)
```

Example : List Contents of a Directory

```
import os
items = os.listdir("my_folder")
print("Contents of directory:", items)
```

Output (Example)

Contents of directory: ['file1.txt', 'file2.png', 'sub_folder', 'data.csv']

Key Point :

It returns a list of files and directories inside the given path.

11. os.getcwd() - Get Current Working Directory

- The `os.getcwd()` function returns the current working directory (the directory from where the script is running).

Syntax

```
os.getcwd()
# returns current working directory
```

Example : Get Current Working Directory

```
import os
cwd = os.getcwd()
print("Current Working Directory:\n", cwd)
```

Output (Example)

Current Working Directory:
C:\Users\User\Documents\Python_Programs

12. os.chdir() - Change Current Working Directory

- The `os.chdir()` function is used to change the current working directory.
- After changing, all relative paths are based on the new directory.

Syntax

```
os.chdir(path)
path → new directory path (string)
```

Example : Change Current Working Directory

```
import os
os.chdir("my_folder")
print("New Working Directory:", os.getcwd())
```

Output (Example)

New Working Directory: C:\Users\User\Documents\Python_Programs\my_folder

How os.chdir() Works ?

Before Changing

Current Working Directory
C:\Users\User\Documents\Python_Programs

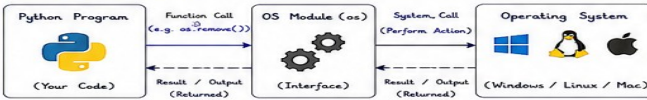
`os.chdir("my_folder")`

After Changing

Current Working Directory
C:\Users\User\Documents\Python_Programs\my_folder

Internal Working of OS Module

Python OS module acts as a bridge between Python program and Operating System.



Common Mistakes

- ✗ Using `os.rmdir()` on a non-empty directory.
- ✗ Using wrong path (relative vs absolute).
- ✗ Forgetting to check existence using `os.path.exists()`.
- ✗ Not handling exceptions (FileNotFoundError, PermissionError, etc.).
- ✗ Using `os.remove()` to a directory (use `os.rmdir()`).

Best Practices

- ✓ Always check if file/directory exists using `os.path.exists()`.
- ✓ Use absolute paths to avoid confusion.
- ✓ Handle exceptions using `try...except`.
- ✓ Use `os.listdir()` to explore directory contents before performing operations.
- ✓ Use `os.getcwd()` to confirm your current working directory.

Quick Reference Table

Function	Purpose	Syntax	Example
<code>os.rename()</code>	Rename a file or directory	<code>os.rename(src, dst)</code>	<code>os.rename("old.txt", "new.txt")</code>
<code>os.remove()</code>	Delete a file	<code>os.remove(path)</code>	<code>os.remove("file.txt")</code>
<code>os.path.exists()</code>	Check if path exists	<code>os.path.exists(path)</code>	<code>os.path.exists("file.txt")</code>
<code>os.path.getsize()</code>	Get size of file (in bytes)	<code>os.path.getsize(path)</code>	<code>os.path.getsize("image.png")</code>
<code>os.mkdir()</code>	Create a directory	<code>os.mkdir(path)</code>	<code>os.mkdir("new_folder")</code>
<code>os.rmdir()</code>	Remove an empty directory	<code>os.rmdir(path)</code>	<code>os.rmdir("empty_folder")</code>
<code>os.listdir()</code>	List directory contents	<code>os.listdir(path)</code>	<code>os.listdir("my_folder")</code>
<code>os.getcwd()</code>	Get current working directory	<code>os.getcwd()</code>	<code>os.getcwd()</code>
<code>os.chdir()</code>	Change current working directory	<code>os.chdir(path)</code>	<code>os.chdir("my_folder")</code>



Remember :

Always handle exceptions and verify paths before performing operations to write safe and reliable Python programs.



Key Takeaway :

The `os` module provides many useful functions to perform file and directory operations and to interact with the operating system easily. It makes our programs powerful and system-friendly.

CHAPTER 12 - SHUTIL MODULE IN PYTHON

1. What is shutil ?

The shutil (shell utilities) module in Python provides high-level operations on files and collections of files. It allows us to copy, move, delete, rename and archive files and directories easily.

2. Why do we need shutil ?

- os module provides basic operations.
- shutil module provides advanced and user-friendly operations.
- Helps in copying entire directories (with subfolders and files) in one command.
- Provides functions for archiving and extracting files.

3. os vs shutil (Quick Difference)

os Module	shutil Module
• Provides low-level functions. • Works file-by-file. • Complex for directory operations.	• Provides high-level functions. • Works on files and entire directories. • Easy and powerful utilities.

4. Importing shutil Module

```
import shutil # import entire module
```

★ **Note :** shutil is a built-in module.
No need to install anything separately.

5. shutil.copy() - Copy a File

- Copies the file to the specified destination.
- Only the file data is copied.
- Metadata (like permissions, timestamps) may not be preserved.

Syntax

```
shutil.copy(src, dst)
```

Example :

```
import shutil
shutil.copy("source.txt", "destination.txt")
print("File copied successfully.")
```

How it Works ?



Key Point :
If destination file exists,
it will be overwritten.

6. shutil.copy2() - Copy a File with Metadata

- Works same as copy(), but also copies metadata.
- Metadata includes : permissions, last modified time, access time, etc.
- Useful when we want an exact copy of the file.

Syntax

```
shutil.copy2(src, dst)
```

Example :

```
import shutil
shutil.copy2("source.txt", "destination.txt")
print("File copied with metadata.")
```

How it Works ?



Metadata includes :

- Permissions
- Creation time
- Modification time
- Access time

7. Difference between copy() and copy2()

Feature	shutil.copy()	shutil.copy2()
What it copies	Only file content (data)	File content + metadata
Metadata (permissions, time etc.)	Not copied	Copied
Use case	When only file data is needed	When exact copy (with metadata) is needed
Speed	Slightly faster	Slightly slower (due to metadata copy)
If destination exists	Overwritten	Overwritten
Example	shutil.copy("a.txt", "b.txt")	shutil.copy2("a.txt", "b.txt")



Remember : Use copy() for simple copying.
Use copy2() when metadata (timestamps, permissions) is important.

CHAPTER 12 - SHUTIL MODULE IN PYTHON

8. shutil.copytree() - Copy an Entire Directory

- Copies an entire directory including all its subdirectories and files.
- Destination directory must not exist (by default).
- Use `dirs_exist_ok=True` (Python 3.8+) to allow copying into an existing directory.

Syntax

```
shutil.copytree(src, dst, dirs_exist_ok=False)
```

Example with `dirs_exist_ok`

```
import shutil
shutil.copytree("my_folder", "backup_folder",
               dirs_exist_ok=True)
print("Directory copied successfully (merged).")
```

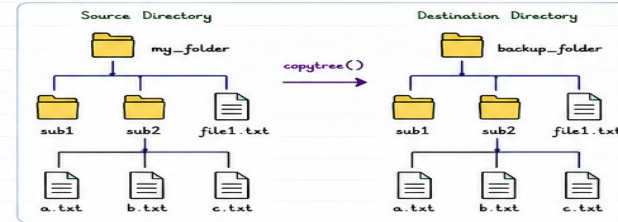
Key Point :

If destination exists and `dirs_exist_ok=False` (default), `FileExistsError` is raised.

Example :

```
import shutil
shutil.copytree("my_folder", "backup_folder")
print("Directory copied successfully.")
```

How it Works ?



9. shutil.move() - Move or Rename Files/Directories

- Moves a file or directory to a new location.
- If the destination is on the same drive, it performs a rename.
- If on a different drive, it performs a copy and then delete.

Syntax

```
shutil.move(src, dst)
```

How it Works ?



Notes :

- ★ If destination is a directory, the source will be moved inside it.
- ★ If destination is a file, source will be renamed to that file.

Examples :

```
# 1. Move a file
import shutil
shutil.move("file.txt", "new_folder/")
print("File moved successfully.")

# 2. Rename a file (same drive)
shutil.move("file.txt", "file_renamed.txt")
print("File renamed successfully.")

# 3. Move a directory
shutil.move("my_folder", "new_location/")
print("Folder moved successfully.")
```

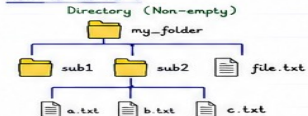
10. shutil.rmtree() - Remove a Directory (Non-empty)

- Removes a directory and all its contents (subdirectories and files).
- Unlike `os.rmdir()`, it can remove non-empty directories.

Syntax

```
shutil.rmtree(path)
```

How it Works ?



Deletes everything
inside the folder
permanently

Directory Removed



Important :

This operation cannot be undone.
Use with care.



Key Takeaway : `shutil.copytree()` copies entire directories, `shutil.move()` moves or renames files/directories, and `shutil.rmtree()` deletes non-empty directories completely.

CHAPTER 12 - SHUTIL MODULE IN PYTHON

11. shutil.make_archive() - Create Archives (ZIP)

- Creates an archive file (zip, tar, gztar, bztar, xztar).
- By default it creates a ZIP file.
- The base_name should not include the extension.
- Returns the name of the archive file.

Syntax

```
shutil.make_archive(base_name, format, root_dir, base_dir=None)
```

Parameters :

- base_name : Name of the archive (without extension)
- format : Archive format ('zip', 'tar', etc.)
- root_dir : Directory to be archived
- base_dir : (Optional) Subdirectory within root_dir

Example : Create ZIP Archive

```
import shutil
archive_name = shutil.make_archive(
    "my_backup", "zip", "my_folder"
)
print("Archive created:", archive_name)
```



12. shutil.unpack_archive() - Extract Archives

- Extracts an archive file to a specified directory.
- It automatically detects the archive format.

Syntax

```
shutil.unpack_archive(filename, extract_dir=None, format=None)
```

Parameters :

- filename : Archive file to extract
- extract_dir : (Optional) Directory to extract to (default: current dir)
- format : (Optional) Archive format (zip, tar, etc.)

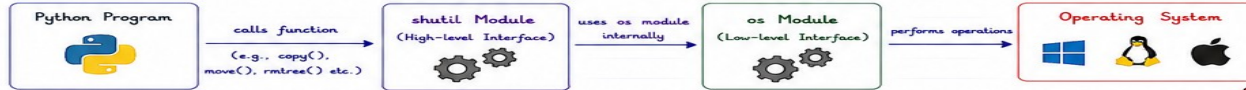
Example : Extract ZIP Archive

```
import shutil
shutil.unpack_archive(
    "my_backup.zip", "extracted_folder"
)
print("Archive extracted successfully:")
```



★ **Note** : If extract_dir is not provided, files will be extracted in the current directory.

13. How shutil Module Works Internally ?



- ✓ shutil is built on top of the os module.
- ✓ It provides user-friendly, high-level functions.
- ✓ Internally, it uses os functions (like open, read, write, rename, etc.) to talk to the OS.
- ✓ Finally, the OS performs the actual file/directory operations.

Example (copy())

shutil.copy() → uses open(), read(), write() → handled by OS → file is copied.

14. os (Basic) vs shutil (Advanced) - Quick Comparison

Feature	os Module (Basic)	shutil Module (Advanced)
Level	Low-level functions	High-level functions
Ease of Use	More code, more steps	Easy, simple, one line
Copy File	Not available directly	shutil.copy(), shutil.copy2()
Copy Directory	Not available	shutil.copytree()
Move / Rename	os.rename() (same drive only)	shutil.move() (works across drives)
Delete Directory	os.rmdir() (only empty dir)	shutil.rmtree() (non-empty dir also)
Create Archive	Not available	shutil.make_archive()
Extract Archive	Not available	shutil.unpack_archive()
Best For	Small, simple tasks	Complex tasks, whole directories, archives

15. Common Mistakes

- ✗ Forgetting dirs_exist_ok=True in shutil.copytree() when destination already exists.
- ✗ Using os.rmdir() on non-empty directory (raises error). Use shutil.rmtree() instead.
- ✗ Trying to use os.rename() to move files across different drives. Use shutil.move() instead.
- ✗ Forgetting to provide correct paths (absolute vs relative).
- ✗ Overwriting important files without backup.

16. Best Practices

- ✓ Always check if source path exists before performing operations.
- ✓ Use shutil.copy2() when file metadata (permissions, timestamps) is important.
- ✓ Use dirs_exist_ok=True with shutil.copytree() if destination may already exist.
- ✓ Use shutil.move() for moving files between different drives.
- ✓ Use shutil.rmtree() to safely delete non-empty directories.
- ✓ Always handle exceptions (try...except) for robust programs.
- ✓ Keep backups before overwriting or deleting files.

17. Summary (Cheat Sheet)

shutil.copy()	→ Copy a file
shutil.copy2()	→ Copy a file with metadata
shutil.copytree()	→ Copy entire directory
shutil.move()	→ Move/Rename file or directory
shutil.rmtree()	→ Delete non-empty directory
shutil.make_archive()	→ Create archive (zip, tar, etc.)
shutil.unpack_archive()	→ Extract archive



Key Takeaway : The shutil module makes file and directory operations easy and powerful. Use it for high-level tasks and let Python handle the complexity for you!



CHAPTER 13 - CSV MODULE IN PYTHON

1. What is CSV ?

- CSV stands for Comma Separated Values.
- It is a simple file format used to store tabular data (rows and columns).
- Each line in CSV file is a row.
- Values in a row are separated by commas (or other delimiters).
- It is widely used in data exchange between applications, spreadsheets, databases, etc.

Example (data.csv)

```
Name,Age,City
Alice,25,Dehradun
Bob,30,Delhi
Charlie,22,Mumbai
```

2. Importing CSV Module

```
import csv # import csv module
```

Note :

CSV files are text files. Always open them with `newline=''` to avoid blank line issues (especially on Windows).

3. Opening a CSV File

Always use `open()` with `newline=''`:

```
f = open('data.csv', mode='r', newline='', encoding='utf-8')
```

Common Modes

Mode	Description
'r'	Read (default)
'w'	Write (overwrites file)
'a'	Append (add at the end)
'x'	Create (fails if file exists)

4. Reading CSV Files

We use `csv.reader()` to read CSV files.

Syntax

```
csv.reader(csvfile, delimiter=',')
```

Example : Read and Print CSV File

```
import csv
with open('data.csv', mode='r', newline='', encoding='utf-8') as file:
    reader = csv.reader(file)
    for row in reader:
        print(row) # each row is returned as a list
```

Output :

```
[['Name', 'Age', 'City']]
[['Alice', '25', 'Dehradun']]
[['Bob', '30', 'Delhi']]
[['Charlie', '22', 'Mumbai']]
```

How it Works ?



Tip :

You can change delimiter if your file uses something else like ; or |
`csv.reader(file, delimiter=';')`

5. Reading CSV with DictReader

It returns each row as a dictionary (key = column name).

Syntax

```
csv.DictReader(csvfile, delimiter=',')
```

Example :

```
import csv
with open('data.csv', mode='r', newline='', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)
```

Output :

```
{'Name': 'Alice', 'Age': '25', 'City': 'Dehradun'}
{'Name': 'Bob', 'Age': '30', 'City': 'Delhi'}
{'Name': 'Charlie', 'Age': '22', 'City': 'Mumbai'}
```

Note :

First row of CSV file is treated as header (column names). DictReader uses this header to create dictionaries.

6. Writing CSV Files

We use `csv.writer()` to write data into CSV files.

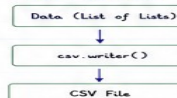
Syntax

```
csv.writer(csvfile, delimiter=',', lineterminator='\n')
```

Example : Write Data to CSV

```
import csv
data = [
    ['Name', 'Age', 'City'],
    ['Alice', '25', 'Dehradun'],
    ['Bob', '30', 'Delhi']
]
with open('output.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

How it Works ?



Note :

`writerow()` → writes a single row
`writerows()` → writes multiple rows

7. Writing CSV with DictWriter

It writes dictionaries into CSV with header.

Syntax

```
csv.DictWriter(csvfile, fieldnames, delimiter=',')
```

Example :

```
import csv
data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'Dehradun'},
    {'Name': 'Bob', 'Age': 30, 'City': 'Delhi'}
]
fields = ['Name', 'Age', 'City']
with open('output2.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.DictWriter(file, fieldnames=fields)
    writer.writeheader() # write header first
    writer.writerows(data)
```

Output File (output2.csv)

```
Name,Age,City
Alice,25,Dehradun
Bob,30,Delhi
```



Important :

Always call `writeheader()` when using DictWriter.

8. Common Parameters

Parameter	Description	Default
delimiter	The character used to separate values.	','
quotechar	Character used to quote fields.	'\"'
lineterminator	Character at end of line.	'\n'
quoting	Controls when quotes are used.	csv.QUOTE_MINIMAL
skipinitialspace	Skip spaces after delimiter.	False

Common quoting values

- `csv.QUOTE_MINIMAL` → Quote only when necessary (default)
- `csv.QUOTE_ALL` → Quote all fields
- `csv.QUOTE_NONNUMERIC` → Quote all non-numeric fields
- `csv.QUOTE_NONE` → Never quote fields

9. Handling Missing / Empty Values

- Empty fields appear as empty strings: ''
- None values are written as empty strings.
- Be careful while converting types (e.g., '25' is string, not integer).

Example :
`row = ['Alice', '', 'Dehradun']`
empty age
Writer output:
Alice,,Dehradun

10. Best Practices

- ✓ Always open CSV files with `newline=''`.
- ✓ Use `encoding='utf-8'` for better compatibility.
- ✓ Use DictReader / DictWriter for working with column names.
- ✓ Close files properly (use with statement).
- ✓ Handle exceptions using try-except
- ✓ Validate data before writing to CSV.



Key Takeaway :

CSV module helps in easily reading and writing tabular data. It is simple, powerful and widely used in real-world applications like data analysis, report generation, exporting/importing data, etc.



CHAPTER 13 - CSV MODULE IN PYTHON



11. Quoting Values

- When writing CSV, some values may contain comma (,), newline (\n), quote ("), etc.
- To avoid confusion, those values are enclosed in double quotes ("").
- The quoting parameter tells Python WHEN to use quotes.

Why Quoting is Needed ?

Suppose we have:

```
data = [["Rahul", "New Delhi, India", 21]]
```

If written without quotes:

```
Rahul,New Delhi, India,21
```

Reader will treat it as 4 columns (Wrong)

If written with quotes:

```
"Rahul","New Delhi, India",21
```

Reader will treat it as 3 columns (Correct)

12. Different Quoting Options

The csv module provides different quoting styles. Use them with `csv.writer()` or `csv.DictWriter()`.

Quoting Option	What Gets Quoted ?	Example Output
csv.QUOTE_MINIMAL (Default)	Only values that need quotes (contains comma, newline, quote, etc.)	Rahul,"New Delhi, India",21
csv.QUOTE_ALL	Every value is quoted	"Rahul","New Delhi, India","21"
csv.QUOTE_NONNUMERIC	Only non-numeric (string) values are quoted	"Rahul","New Delhi, India",21
csv.QUOTE_NONE	No values are quoted (Use escapechar for safety)	Rahul,New Delhi, India,21 (Risky: extra columns)



Note :

If quoting=csv.QUOTE_NONE, you must provide escapechar, otherwise an error will occur if delimiter appears inside data.

Example: `csv.writer(f, quoting=csv.QUOTE_NONE, escapechar='\\')`

13. Reading CSV with csv.reader()

- `csv.reader()` reads each row as a list.
- Best when you only need data without column names.

Syntax

```
csv.reader(csvfile, delimiter=',', quotechar='"',  
lineterminator='\n', quoting=csv.QUOTE_MINIMAL)
```

Example :

```
import csv  
with open('students.csv', mode='r', newline='') as file:  
    reader = csv.reader(file)  
    for row in reader:  
        print(row) # each row is returned as a list
```

Output (students.csv)

Name	Age	City
Alice	25	Dehradun
Bob	30	Delhi
Charlie	22	Mumbai



Tip :

Always use `newline=''` while opening CSV file to avoid extra blank lines (especially on Windows).

14. Reading CSV with DictReader

- `csv.DictReader()` reads CSV into a dictionary.
- First row of CSV is treated as header (column names).
- Best when working with structured data.

Syntax

```
csv.DictReader(csvfile, delimiter=',', quotechar='"',  
lineterminator='\n', quoting=csv.QUOTE_MINIMAL)
```

Example :

```
import csv  
with open('students.csv', mode='r', newline='') as file:  
    reader = csv.DictReader(file)  
    for row in reader:  
        print(row)  
        print(row['Name'], row['Age'], row['City'])
```

Output :

Row as Dictionary

```
{'Name': 'Alice', 'Age': '25', 'City': 'Dehradun'}  
{'Name': 'Bob', 'Age': '30', 'City': 'Delhi'}  
{'Name': 'Charlie', 'Age': '22', 'City': 'Mumbai'}
```

Access Individual Values

```
Name 25 Dehradun  
Bob 30 Delhi  
Charlie 22 Mumbai
```

15. Writing CSV with DictWriter

- `csv.DictWriter()` writes dictionaries into CSV.
- We must provide fieldnames (header names).

Syntax

```
csv.DictWriter(csvfile, fieldnames, delimiter=',',  
quotechar='"', lineterminator='\n',  
quoting=csv.QUOTE_MINIMAL)
```

Example :

```
import csv  
data = [  
    {'Name': 'Alice', 'Age': 25, 'City': 'Dehradun'},  
    {'Name': 'Bob', 'Age': 30, 'City': 'Delhi'},  
    {'Name': 'Charlie', 'Age': 22, 'City': 'Mumbai'}  
,  
    ]  
with open('output.csv', mode='w', newline='') as file:  
    fields = ['Name', 'Age', 'City'] # header  
    writer = csv.DictWriter(file, fieldnames=fields)  
    writer.writeheader() # write header first  
    writer.writerows(data) # write all rows
```

Output File (output.csv)

Name	Age	City
Alice	25	Dehradun
Bob	30	Delhi
Charlie	22	Mumbai



Important :

Always call `writeheader()` before `writerows()`, otherwise header will not be written.

16. Summary (Quick View)

Task	Function / Class
Read CSV as list	<code>csv.reader()</code>
Read CSV as dict	<code>csv.DictReader()</code>
Write CSV	<code>csv.writer()</code>
Write CSV with header (dict)	<code>csv.DictWriter()</code>
Control quoting	quoting parameter
Specify delimiter	delimiter parameter
Avoid blank lines	<code>open(..., newline='')</code>



Key Takeaway :

CSV is simple, lightweight and widely used for tabular data. Use `reader/DictReader` for reading and `writer/DictWriter` for writing.



CHAPTER 13 - CSV MODULE IN PYTHON



17. Using CSV with Pandas (Quick)

- Pandas makes working with CSV files very easy.
- It is not part of built-in csv module, but widely used in data analysis.

Example :

```
import pandas as pd
# Read CSV
df = pd.read_csv('students.csv')
print(df)
# Write CSV
df.to_csv('new_students.csv', index=False)
```

students.csv

Name	Age	City
Alice	25	Dehradun
Bob	30	Delhi
Charlie	22	Mumbai

Pandas DataFrame



Note : Pandas is built on top of CSV for high level data manipulation.

19. Complete Example (Read → Modify → Write)



```
import csv
# Step 1 : Read CSV
with open('students.csv', mode='r', newline='', encoding='utf-8') as file:
    reader = csv.DictReader(file)
    data = list(reader) # read all rows as list of dictionaries

# Step 2 : Modify Data (e.g. increase age by 1)
for row in data:
    row['Age'] = str(int(row['Age']) + 1)

# Step 3 : Write to New CSV
with open('updated_students.csv', mode='w', newline='', encoding='utf-8') as file:
    fieldnames = ['Name', 'Age', 'City']
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

Original (students.csv)

Name	Age	City
Alice	25	Dehradun
Bob	30	Delhi
Charlie	22	Mumbai

Updated (updated_students.csv)

Name	Age	City
Alice	26	Dehradun
Bob	31	Delhi
Charlie	23	Mumbai

20. Delimiter Examples

CSV is not limited to comma (,). You can use any delimiter.

Delimiter	Code Example	Output Example
Comma (,)	delimiter=','	a,b,c
Semicolon (;)	delimiter=';'	a;b;c
Tab (\t)	delimiter='\t'	a b c
Pipe ()	delimiter=' '	a b c



Tip : Choose delimiter that does not appear in your data. Comma is default and most commonly used.

21. Dialect in CSV

- A dialect is a collection of CSV formatting parameters.
- csv module provides some built-in dialects.

Built-in Dialects

Dialect	Description
excel	Default dialect (comma, '\r\n')
excel-tab	Tab delimiter
unix	Comma delimiter, '\n' as line terminator

Example :

```
import csv
with open('data.csv', newline='') as file:
    reader = csv.reader(file, dialect='excel-tab')
```

22. Sniffer Object (Auto-detect Dialect)

csv.Sniffer can detect delimiter and quotechar from a sample of data. Useful when you don't know file format.

Example :

```
import csv
with open('unknown.csv', newline='') as file:
    sample = file.read(1024)
    file.seek(0) # go back to start
    sniffer = csv.Sniffer()
    dialect = sniffer.sniff(sample)
    reader = csv.reader(file, dialect)
    for row in reader:
        print(row)
```



23. Quick Cheat Sheet

Task	Code / Function	Task	Code / Function
Open CSV File	open('file.csv', newline='', encoding='utf-8')	Write Header (DictWriter)	writeheader()
Read as List	csv.reader(file)	Write Multiple Rows	writerows(list_of_rows)
Read as Dict	csv.DictReader(file)	Specify Delimiter	delimiter=', ' (or other)
Write Data	csv.writer(file)	Specify Quote Style	quoting=csv.QUOTE_MINIMAL
Write Dict Data	csv.DictWriter(file, fieldnames=[...])	Avoid Blank Lines	open(file, newline='')

★ Final Note

CSV files are simple, readable and platform independent. Always handle newline, encoding, and quoting properly for clean and robust CSV operations.



CHAPTER 13 - CSV MODULE IN PYTHON



32. Working with Large CSV Files

- CSV module is memory efficient.
- It reads and writes data row by row, so it can handle large files easily.

Example : Read large file row by row

```
import csv
with open('large_file.csv', 'r', newline='') as f:
    reader = csv.reader(f)
    for i, row in enumerate(reader, start=1):
        if i % 100000 == 0:
            print(f"Read {i} rows...")
```



Tip :

Process one row at a time instead of loading the entire file into memory.

33. CSV with Different Encodings

- Use encoding parameter while opening file.
- Useful when CSV contains special characters.

Example :

```
import csv
with open('data.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```



Common encodings :

- 'utf-8' (default and recommended)
- 'latin-1' (used in older systems)
- 'cp1252' (common in Windows)

34. CSV with Headers (Column Names)

- First row is usually the header.
- Use it for better readability and access.

Example : Using DictReader

```
import csv
with open('employees.csv', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['Salary'])
```

employees.csv

Name	Department	Salary
Alice	HR	50000
Bob	IT	60000
Charlie	Finance	55000

35. CSV Without Header (Add Your Own)

Example : Writing CSV without header

```
import csv
data = [
    ['Alice', 'HR', 50000],
    ['Bob', 'IT', 60000]
]
with open('no_header.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(data)
```



Note :

If there is no header, you must remember the order of columns while reading the file.

36. Append Data to CSV File

Use mode='a' to append data at the end.

Example :

```
import csv
new_row = ['Diana', 'IT', 62000]
with open('employees.csv', 'a', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(new_row)
```

37. Update CSV File (Rewrite)

- CSV files do not support direct update.
- We read all data, modify it, then write again.

Steps :

1. Read all rows
2. Modify data
3. Write back all rows

Example : Increase salary by 10%

```
import csv
rows = []
with open('employees.csv', newline='') as f:
    reader = csv.DictReader(f)
    fieldnames = reader.fieldnames
    for row in reader:
        row['Salary'] = int(row['Salary']) * 110 // 100
        rows.append(row)
with open('employees.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)
```

38. Delete Row(s) in CSV

- Remove unwanted rows using conditions.
- Then write remaining rows back.

Example : Delete employee named 'Bob'

```
import csv
rows = []
with open('employees.csv', newline='') as f:
    reader = csv.DictReader(f)
    fieldnames = reader.fieldnames
    for row in reader:
        if row['Name'] != 'Bob':
            rows.append(row)
with open('employees.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)
```

39. Filter Data from CSV

Example : Get employees from IT department

```
import csv
with open('employees.csv', newline='') as f:
    reader = csv.DictReader(f)
    print('Name\tSalary')
    for row in reader:
        if row['Department'] == 'IT':
            print(row['Name'], '\t', row['Salary'])
```



40. Sort CSV Data

Example : Sort by Salary (Ascending)

```
import csv
with open('employees.csv', newline='') as f:
    reader = csv.DictReader(f)
    rows = list(reader)
rows.sort(key=lambda x: int(x['Salary']))
with open('employees_sorted.csv', 'w', newline='') as f:
    writer = csv.DictWriter(f, fieldnames=reader.fieldnames)
    writer.writeheader()
    writer.writerows(rows)
```



41. Merge (Combine) CSV Files

Example : Merge two CSV files

```
import csv
with open('file1.csv', newline='') as f1, \
    open('file2.csv', newline='') as f2, \
    open('merged.csv', 'w', newline='') as out:
    reader1 = csv.reader(f1)
    reader2 = csv.reader(f2)
    writer = csv.writer(out)
    writer.writerow(reader1)
    writer.writerow(reader2)
```



42. Helpful CSV Tips

- ✓ Always open CSV files with newline=''
- ✓ Use utf-8 encoding for international data.
- ✓ Use DictReader/DictWriter for working with headers.
- ✓ Use QUOTE_MINIMAL (default) for general use.
- ✓ Always close files (use with statement).
- ✓ Validate and clean data before writing to CSV.



43. Common CSV File Extensions

Extension	Description
.csv	Comma Separated Values (most common)
.txt	Text file (can store CSV format)
.tsv	Tab Separated Values
.psv	Pipe Separated Values
.dat	Data file (custom)

44. CSV Reader & Writer Quick View

Mode	Class / Function	Purpose
Read	csv.reader()	Read as list
	csv.DictReader()	Read as dictionary
Write	csv.writer()	Write rows (list)
	csv.DictWriter()	Write rows (dict)



Key Takeaway :

The csv module is simple but very powerful for handling tabular data. Master reader, writer, DictReader, DictWriter and quoting to work efficiently.

